Prism Chat App — Interview Guide

Oct 27, 2026 · @Amitabha Banerjee

1. Project overview and the 60-second pitch

Prism is a full-stack, real-time 1:1 chat app on the MERN stack. Messages and image URLs are end-to-end encrypted in the browser with TweetNaCl (`nacl.box`), so the server stores and relays only ciphertext. Everything in this guide comes from the actual code in this repo, so you can defend every answer.

The pitch (memorise this)

> "Prism is a real-time, end-to-end encrypted chat app I built with React 19, Zustand, Node/Express 5, MongoDB and Socket.IO. When you sign up, your browser generates a Curve25519 key pair. The public key goes to the server and the secret key never leaves your device. Every message is encrypted client-side with the recipient's public key, using a fresh random nonce, so MongoDB only ever holds ciphertext. Delivery is instant over WebSockets, with online presence, unread counts and read receipts. You can sign in on a new device by pasting your secret key, and you can see and revoke devices. Auth uses JWTs in httpOnly cookies. It's deployed on Render as a single service: Express serves both the API and the built React app."

Tech stack at a glance

Layer	Technology	Where in code	Why it was chosen
UI	React 19 + Vite 8	`frontend/src`	Fast dev server, modern React
Styling	Tailwind CSS v4 + daisyUI v5	`index.css`, `constants/index.js` (9 themes)	Utility-first styling, themes via `data-theme`
State	Zustand 5	`stores/useAuthStore.js`, `useChatStore.js`, `useThemeStore.js`	Tiny, no boilerplate, readable outside React (`getState()`)

Layer	Technology	Where in code	Why it was chosen
Routing	React Router v7	`App.jsx`	Protected routes; modals mapped to paths
HTTP	Axios (`withCredentials: true`)	`lib/axios.js`	Sends the JWT cookie on every request
Crypto	tweetnacl + tweetnacl-util	`lib/crypto.js`	Audited, small, simple API (X25519 + XSalsa20-Poly1305)
Realtime	Socket.IO 4.8	`backend/src/lib/Socket.js`	Rooms/emit API, auto-reconnect, long-polling fallback
Server	Node + Express 5	`backend/src/index.js`	Simple REST, native async error handling
DB	MongoDB Atlas + Mongoose 9	`models/user.model.js`, `messages.model.js`	Flexible documents, embedded devices/contacts
Auth	JWT (7 days) + bcrypt (10 rounds)	`lib/utils.js`, `auth.middleware.js`	Stateless sessions; slow password hashing
Media	Cloudinary	`lib/cloudinary.js` (avatars), client unsigned upload (chat images)	CDN hosting, no file storage on the server
Notifications	react-hot-toast, Audio API	`App.jsx`, `useChatStore.js`	Toasts; 3 selectable sounds + mute
Hosting	Render	root `package.json` `build` / `start`	One service for API + static frontend

Feature list (be ready to demo each one)

1. Sign up / log in / log out with JWT in an httpOnly, SameSite=strict cookie.

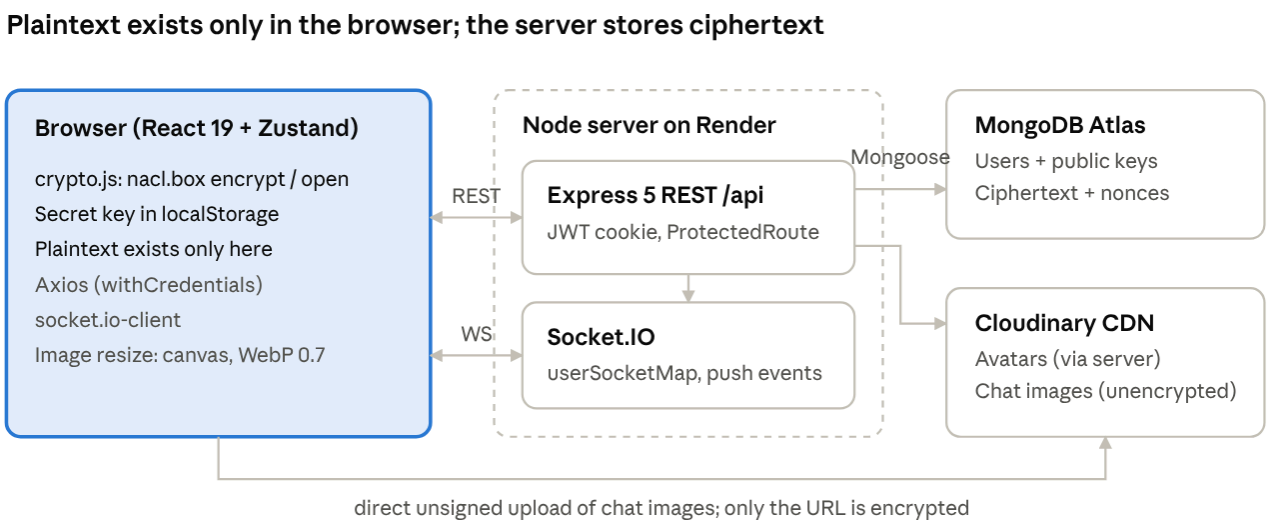
2. Key pair generated in the browser at sign-up; secret key saved in `localStorage` as `<email>_secret_key`.
3. E2E-encrypted text messages and image URLs (`text` + `textNonce`, `image` + `imageNonce`).
4. Real-time delivery over Socket.IO (`newMessage` event).
5. Online presence (`getOnlineUsers` broadcast) plus an "Online only" filter.
6. Unread counts per conversation, and read receipts (`isRead`, `messagesRead` event).
7. Contacts: find by exact name or email, add, delete. The sidebar shows contacts plus anyone who has messaged you.
8. Multi-device: "Unlock this device" by pasting the secret key (checked by re-deriving the public key). Devices list with per-device remove; removing the current device wipes its local key.
9. Image messages: resized client-side to at most 1000 px, WebP at 70% quality, uploaded to Cloudinary, URL encrypted.
10. Profile picture upload (base64 → server → Cloudinary).
11. 9 glassmorphism themes (Obsidian, Midnight, Aurora, and more), a theme-coloured dynamic favicon, legacy theme migration.
12. Chat UX: messages grouped within a 5-minute window, day dividers ("Today", "Yesterday"), image lightbox (Esc closes it), skeleton loaders, mobile layout with back buttons.

Numbers to quote

- About 2,200 lines of JSX in pages/components; 2 Mongoose models; 16 REST endpoints across 2 routers; 3 socket events.
- JWT lifetime 7 days; bcrypt cost 10; minimum password length 6; JSON body limit 50 MB; image max 1000×1000, WebP q=0.7; message grouping window 5 minutes.

2. Architecture deep dive

Prism is a monorepo with two apps: `frontend/` (Vite SPA) and `backend/` (Express + Socket.IO on one HTTP server). In production, Express serves `frontend/dist` and falls back to `index.html` for every non-API route. One Render service hosts everything on one origin, which is also why the CORS config can hard-code `http://localhost:5173`: that's only needed in development.



Prism architecture · browser, one Node server, two cloud services

Encryption and decryption happen only in the browser. Express and Socket.IO share one HTTP server, and chat images skip the server and go straight to Cloudinary.

Folder map

Path	Responsibility
`backend/src/index.js`	Loads env, CORS, `express.json({limit:'50mb'})`, cookie-parser, mounts `/api/auth` and `/api/messages`, serves the SPA in production, `server.listen` then `connectDB()`
`backend/src/lib/Socket.js`	Creates `app`, `http.createServer(app)`, the Socket.IO `Server`; holds the `userSocketMap` {userId: socketId}; exports `getReceiverSocketId`
`backend/src/lib/utils.js`	`generateToken`: signs `{userId}` with `JWT_SECRET`, 7-day expiry, sets the `jwt` cookie
`backend/src/middleware/auth.middleware.js`	`ProtectedRoute`: reads the cookie, `jwt.verify`, loads the user without the password, attaches `req.user`
`backend/src/controllers/*`	Business logic for auth/contacts/devices and messages

Path	Responsibility
`frontend/src/stores/*`	Zustand stores: auth + socket, chat + crypto, theme + sound
`frontend/src/lib/crypto.js`	`generateKeyPair`, `verifySecretKey`, `encryptText`, `decryptText`
`frontend/src/lib/utils.js`	Time formatting, day labels, `getDeviceInfo()` (UA parsing, device UUID kept in localStorage)

REST API (16 endpoints)

Method	Path	Auth	What it does
POST	`/api/auth/signup`	No	Validates input, hashes the password, stores `publicKey` + first device, sets the cookie, returns 201
POST	`/api/auth/login`	No	bcrypt compare, sets the cookie (returns 201; 200 would be more correct)
GET	`/api/auth/logout`	No	Clears the cookie (`maxAge: 0`)
PUT	`/api/auth/updateProfile`	Yes	Uploads a base64 image to Cloudinary, saves `secure_url`
GET	`/api/auth/check`	Yes	Returns `req.user`; the app calls it on load
POST	`/api/auth/addContact`	Yes	`$push` of the contact object onto `contacts`
GET	`/api/auth/getContacts`	Yes	Returns `user.contacts`
DELETE	`/api/auth/delete/:id`	Yes	`$pull` a contact (see the bug in section 7)
POST	`/api/auth/device`	Yes	Registers a device unless that `deviceId` already exists
GET	`/api/auth/device`	Yes	Lists devices
DELETE	`/api/auth/device/:deviceId`	Yes	Filters out the device and saves
GET	`/api/messages/users`	Yes	All users except me (used by Add Contact search)

Method	Path	Auth	What it does
GET	`/api/messages/sidebar-users`	Yes	Contacts ∪ people who messaged me, plus `unreadCounts`
GET	`/api/messages/:id`	Yes	The conversation with user `:id`, sorted by `createdAt` ascending
POST	`/api/messages/send/:id`	Yes	Saves the ciphertext message and emits `newMessage` to the receiver's socket
PUT	`/api/messages/mark-read/:id`	Yes	`updateMany isRead=true`, emits `messagesRead` to the sender

Route order matters: `/users` and `/sidebar-users` are declared before `/:id`, otherwise Express would match them as an id.

Socket.IO events

Event	Direction	Payload	Purpose
`connection`	client → server	`handshake.query.userId`	Map userId → socket.id
`getOnlineUsers`	server → all	array of userIds	Presence; sent on every connect/disconnect
`newMessage`	server → receiver	the saved Message (ciphertext)	Live delivery
`messagesRead`	server → sender	`{receiverId}`	Read receipts
`disconnect`	built-in	—	Remove from the map, re-broadcast presence

Data model

User: `email` (unique), `fullName`, `password` (bcrypt hash), `profilePic` (Cloudinary URL), `publicKey` (base64 Curve25519 key), `contacts` (untyped `Array` of user snapshots), `devices` (sub-documents: `deviceId`, `deviceType`, `deviceName`, `browser`, `os`, `ipAddress`, `lastActive`), timestamps.

Message: `senderId` and `receiverId` (ObjectId refs to User), `text` (base64 ciphertext), `image` (base64 ciphertext of the Cloudinary URL), `textNonce`, `imageNonce`, `isRead` (default false), timestamps. Index: `{senderId: 1, receiverId: 1}`.

Why embed devices and contacts? They are small, bounded, always read along with the user, and need no joins. Why keep messages in their own collection? They are unbounded; embedding them would hit MongoDB's 16 MB document limit.

Flow: sign-up

1. `SignUpPage` validates name, an email regex, and a password of at least 6 characters.
2. `useAuthStore.signup` calls `generateKeyPair()` and saves the secret key to `localStorage[email_secret_key]`.
3. `getDeviceInfo()` builds the device record (random UUID kept in localStorage as `deviceId`).
4. POST `/auth/signup` with `{fullName, email, password, publicKey, devices: [device]}`.
5. The server checks for duplicates, hashes with bcrypt cost 10, saves, sets the JWT cookie, returns the user.
6. The client sets `authUser` and calls `connectSocket()`, which registers the global listeners.

Flow: sending a message

1. `MessageInput`: an optional image is resized on a canvas (max 1000 px) and turned into a WebP data URL.
2. `sendMessage` reads my secret key from localStorage and the receiver's `publicKey` from `selectedUser`.
3. If there is an image, it goes to Cloudinary (unsigned preset); the returned `secure_url` is encrypted with `encryptText`.
4. The text is encrypted the same way, with a fresh 24-byte nonce from `nacl.randomBytes`.
5. POST `/messages/send/:id`. The server saves the ciphertext and looks up `userSocketMap[receiverId]`. If the receiver is online, it calls `io.to(socketId).emit('newMessage', msg)`.
6. The sender appends the server response to `messages`, swapping in the plaintext text and the local image (so the sender doesn't need to decrypt it again).
7. The receiver's `newMessage` handler: if that chat is open, it decrypts, appends, plays the sound, and calls `mark-read`. Otherwise it bumps `unreadCounts[senderId]` and plays the sound.

Flow: read receipts

1. Opening a chat (`setSelectedUser`) sets the local unread count to 0 and calls PUT `/messages/mark-read/:id` .
2. The server runs `updateMany` and emits `messagesRead {receiverId: me}` to the sender's socket.
3. The sender's store marks `isRead: true` on its messages to that receiver, so the UI shows "seen" on the last sent message.

Flow: new device login

1. Log in works normally (password + cookie), but the local key is missing.
2. `HomePage` 's effect sees there is no `<email>_secret_key` and navigates to `/setSecretKey` . A modal that can't be dismissed asks for the key.
3. `verifySecretKey` : `nacl.box.keyPair.fromSecretKey(sk)` re-derives the public key and compares it to `authUser.publicKey` .
4. If they match, the key is stored and the device is registered with POST `/auth/device` .
5. A second `HomePage` effect checks on every mount that this device is still in the server's device list. If it isn't (it was revoked elsewhere), it deletes the local key and forces re-entry.

Flow: app boot / session restore

`App` calls `checkAuth()` (GET `/auth/check`) while showing a full-screen loader (`isCheckingAuth`). On success it sets `authUser` and connects the socket. On a 401, `authUser` stays null and routes redirect to `/login` . Modal pages (`/add-Contacts` , `/setSecretKey` , `/secretKey` , `/devices`) render the Home or Profile page underneath and overlay a `GlassModal` based on `location.pathname` .

3. Feature-by-feature questions and model answers

Each answer is short enough to say aloud in about 30 seconds. Bold lines are the questions; the text under them is what to say.

3.1 End-to-end encryption (the question they will ask most)

Q: How does your E2EE work, step by step? At sign-up the browser runs `nacl.box.keyPair()` , which makes a Curve25519 key pair. The public key (base64) is sent to the server and stored on the User. The secret key stays in `localStorage` . To send, I call `nacl.box(message, nonce, receiverPublicKey, mySecretKey)` . Internally that does an X25519 Diffie-Hellman to get a shared key, then XSalsa20 encryption with a Poly1305 MAC.

The 24-byte nonce comes from `nacl.randomBytes`, and the ciphertext and nonce are both stored. The receiver calls `nacl.box.open` with my public key and their secret key; both sides derive the same shared secret.

Q: How does the sender read their own sent messages if they were encrypted for the receiver? Diffie-Hellman is symmetric: DH(mySecret, theirPublic) = DH(theirSecret, myPublic). So I can decrypt my own sent messages with the other person's public key and my secret key. That's exactly what `getMessages` does for every message in the conversation, using `selectedUser.publicKey` and my secret key.

Q: What is a nonce and why must it be unique? A "number used once". XSalsa20 is a stream cipher; reusing a nonce with the same key reuses the keystream. Then XOR-ing two ciphertexts leaks the XOR of the plaintexts, and Poly1305 forgery becomes possible. The nonce is 24 bytes (192 bits), so random generation has a negligible collision risk. That's why XSalsa20 (extended nonce) is used rather than Salsa20.

Q: Is the nonce secret? No. It's stored next to the ciphertext (`textNonce`). It only needs to be unique.

Q: What does Poly1305 give you? Authenticity and integrity. If the server tampers with the ciphertext, `nacl.box.open` returns `null` and my code shows `[Encrypted Message]`. Because the box uses the sender's key, it also proves the message came from someone holding that sender's secret key.

Q: Why TweetNaCl and not the Web Crypto API? It's small, audited, has no dependencies, and its API is hard to misuse (one call to box, one to open). Web Crypto in 2024–25 doesn't have X25519 in every browser, and building ECDH + AES-GCM + key import/export by hand is more code and more ways to make mistakes. The trade-off: TweetNaCl runs in JS, not native code, which is fine for short messages.

Q: Why is the message image only an encrypted URL? Honest answer: the file is uploaded to Cloudinary unencrypted, and only its URL is encrypted in the database. The server operator and Mongo can't see which image belongs to which message, but anyone who gets the URL can open the image. The proper fix is to encrypt the image bytes with `nacl.secretbox` and a random key, upload that blob, and send the key inside the encrypted message. That's how Signal/WhatsApp attachments work.

Q: How do you prove the server never sees plaintext? The README screenshots show the database ciphertext next to the UI. In code, `sendMessage` only posts `encryptedText` and `nonce`. The server's `sendMessage` controller stores what it receives and never touches keys.

Q: Can the server still attack you? Yes, with a man-in-the-middle on key distribution. The server hands out public keys, so a malicious server could give Alice its own public key instead of Bob's. Mitigation: safety numbers / key fingerprints that users compare out of

band (Signal's QR codes), or key transparency logs. Prism doesn't have that yet, so its E2EE trusts the server for key distribution ("trust on first use" without any warning on key change).

Q: Do you have forward secrecy? No. Keys are long-lived static key pairs, so a stolen secret key decrypts every past and future message. Signal fixes this with X3DH plus the Double Ratchet: one-time prekeys and a fresh key for each message. I'd name that as the next step.

Q: What metadata is still visible to the server? Who talks to whom (`senderId` / `receiverId`), when (`createdAt`), message size, whether there's an image, read status, IP addresses, online presence, contact lists.

Q: Where is the secret key stored and what are the risks? In `localStorage` under `<email>_secret_key`. Risks: any XSS can read it (localStorage is readable by JS), it's not encrypted at rest, and clearing site data loses it. Better: store it in IndexedDB as a non-extractable `CryptoKey`, or encrypt it with a key derived from the password (PBKDF2/Argon2), or sync an encrypted key backup to the server (WhatsApp's encrypted backups).

Q: What happens if a user loses their secret key? Old messages can't be decrypted. There's no recovery, by design; the server can't help. The Secret Key page tells users to save it in a password manager. Possible improvement: encrypted cloud backup protected by a passphrase, or a recovery phrase (BIP-39 style).

Q: How does "Unlock this device" validate a pasted key without the server? `verifySecretKey` decodes the key, runs `nacl.box.keyPair.fromSecretKey`, and compares the derived public key with `authUser.publicKey`. A Curve25519 public key is a deterministic function of the secret key, so a match proves the key is correct. The secret key is never sent anywhere.

Q: Why is the key keyed by email in localStorage? So several accounts on the same browser don't overwrite each other's keys.

Q: What would group chat E2EE need? Either encrypt the message once per member (simple, O(n) work), or use sender keys / MLS: one symmetric group key, distributed to each member through pairwise boxes and rotated when membership changes.

3.2 Real-time messaging and presence

Q: How does a message reach the receiver instantly? The sender POSTs over REST; the controller saves it, then `getReceiverSocketId(receiverId)` looks it up in the in-memory `userSocketMap`. If the receiver is online, `io.to(socketId).emit('newMessage', newMessage)` sends it. The client's `initGlobalListeners` handler decrypts and appends it.

Q: Why send over REST and not `socket.emit`? REST gives a request/response with a status code, so the sender knows the message was saved, error handling is simple, and the auth middleware is reused. The socket is only used for server-to-client pushes. This is a common hybrid pattern. The alternative is socket emits with acknowledgement callbacks.

Q: How does presence work? On `connection` the server stores `userSocketMap[userId] = socket.id` and broadcasts `getOnlineUsers` with `Object.keys(userSocketMap)` to everyone. On `disconnect` it deletes the entry and broadcasts again. The client stores `onlineUsers` and shows green dots and the "Online only" filter.

Q: What happens with two tabs or two devices? It's a known limitation. The map holds one socket id per user, so the last connection wins. Only that tab gets `newMessage`, and when any one tab disconnects the user shows as offline. Fix: map userId to a Set of socket ids, or better, `socket.join(userId)` and `io.to(userId).emit(...)`, which sends to every socket in that user's room.

Q: What if the receiver is offline? Nothing is emitted. The message is already in MongoDB, so it loads on their next `getMessages`, and `getSidebarUsers` computes unread counts from `isRead: false`. There are no push notifications yet (you'd add Web Push / FCM).

Q: Why WebSockets instead of polling? Fewer requests, lower latency, and the server can push. Socket.IO adds auto-reconnect, heartbeats, a long-polling fallback, and rooms. Trade-off: stateful connections make horizontal scaling harder (see section 5).

Q: Socket.IO vs raw WebSocket? Socket.IO is a protocol on top of WebSocket. It gives named events, acknowledgements, rooms, namespaces, reconnection and fallbacks. It isn't wire-compatible with a plain `ws` client. Raw `ws` is lighter but you build all of that yourself.

Q: How do you prevent duplicate socket listeners? `initGlobalListeners` calls `socket.off('newMessage')` before `socket.on(...)`. `connectSocket` returns early if a socket is already connected. The chat store is loaded with a dynamic `import()` to avoid a circular import between the two stores.

Q: How are notification sounds played? `new Audio('/sounds/<choice>.mp3').play()`, skipped when muted. Browsers block autoplay until the user has interacted with the page, so the `.catch` logs the error instead of crashing.

3.3 Read receipts and unread counts

Q: How are unread counts computed? On the server, `getSidebarUsers` finds my messages with `isRead: false` and counts them per `senderId`. On the client, a `newMessage` from someone whose chat isn't open increments `unreadCounts[senderId]`,

and opening the chat resets it to 0.

Q: How do read receipts work? Opening a chat, or receiving a message while that chat is open, calls PUT `mark-read/:senderId`. The server runs `updateMany({senderId, receiverId: me, isRead: false}, {$set: {isRead: true}})` and emits `messagesRead` to the sender, whose UI flips `isRead` on those messages.

Q: Is the unread computation efficient? Not at scale: it loads every unread message document. Better: an aggregation `$match {receiverId, isRead:false} → $group {_id:'$senderId', count:{$sum:1}}`, backed by an index on `{receiverId:1, isRead:1}`, or a per-conversation counter.

3.4 Contacts and sidebar

Q: How is the sidebar built? It's the union of my contacts' ids and the ids of everyone who has messaged me, de-duplicated with a `Set` and loaded with `$in`, without passwords. Strangers who message you therefore appear, with an "Add" button in the header.

Q: How does Add Contact search work? The client fetches all users (`/messages/users`) and finds an exact, case-insensitive match on name or email. It's simple but doesn't scale and lets anyone list every user. Better: a server-side search endpoint matching email exactly, rate-limited, returning minimal fields.

Q: Why store contacts as embedded objects? It was quick to build, but it's a denormalisation trade-off: a copy of the profile (name, picture, publicKey) goes stale. A better design stores only ObjectIds (`[{type: ObjectId, ref:'User'}]`) and uses `populate`, or a separate Contacts collection.

3.5 Devices

Q: How is a device identified? `getDeviceInfo()` makes a `crypto.randomUUID()` once and keeps it in localStorage as `deviceId`. Browser and OS are parsed from `navigator.userAgent`, and the type is Mobile or Desktop based on a regex.

Q: What does removing a device actually do? It deletes the device sub-document. If it's the current device, the client also deletes its secret key and `deviceId` and logs out. Another device that was removed notices on its next `HomePage` mount (device not in the list), wipes its local key, and must re-enter it. The UI hides "Remove" when only one device is left.

Q: Is that real revocation? Partly. It's enforced on the client: the removed device's JWT is still valid for up to 7 days, and the secret key itself isn't rotated, so someone who copied it can still decrypt. Real revocation would need server-side sessions, or a token version / denylist checked in `ProtectedRoute`, and a key rotation with re-encryption, or per-device keys.

Q: Why is `ipAddress` empty? The client can't reliably know its public IP. It should be filled on the server from `req.ip` / `x-forwarded-for` (with `app.set('trust proxy', 1)` behind Render's proxy).

3.6 Images and profile pictures

Q: How are chat images optimised? The file is read with `FileReader`, drawn on a `<canvas>` scaled to fit within 1000×1000 while keeping the aspect ratio, and exported with `toDataURL('image/webp', 0.7)`. A typical phone photo drops from several MB to about 100–200 KB.

Q: Why do chat images upload directly from the browser, while avatars go through the server? Uploading direct to Cloudinary with an unsigned preset keeps the large binary off my server and away from Express's body limit. Avatars go through the server, which uses the API secret, so the URL is saved only after an authenticated upload. Trade-off: anyone who has the unsigned preset name can upload to my Cloudinary account. A signed upload (the server issues a short-lived signature) fixes that.

Q: Why `express.json({limit:'50mb'})`? The base64 avatar goes in the JSON body, and base64 adds about 33% size. 50 MB is too generous, though (a DoS risk); 5–10 MB, or a multipart upload with multer, would be better.

3.7 Auth

Q: Walk me through login. POST email + password, `User.findOne({email})`, `bcrypt.compare`. If either the user or the password is wrong, the response is the same "Invalid Credentials", so attackers can't tell which accounts exist. Then `generateToken` signs `{userId}` (7 days) and sets the `jwt` cookie with `httpOnly`, `sameSite:'strict'`, and `secure` outside development.

Q: How are protected routes enforced? On the server, `ProtectedRoute` middleware on every private endpoint. On the client, React Router elements render `<Navigate to='/login'/>` when `authUser` is null. Client-side guards are only for UX; the server is the real check.

Q: Why cookies instead of localStorage for the JWT? httpOnly cookies can't be read by JavaScript, so XSS can't steal the token. SameSite=strict stops cross-site requests from carrying it (CSRF). Axios `withCredentials: true` sends the cookie, and in development CORS needs `credentials: true` and an explicit origin.

Q: How does logout work, and what are its limits? It overwrites the cookie with `maxAge: 0`. Because JWTs are stateless, a token copied earlier stays valid until it expires. Fixes: short-lived access tokens plus a refresh token stored in the DB, or a `tokenVersion` on the user.

3.8 UI and UX

Q: How do you group messages? In a `useMemo` over `messages`, each row computes `startsNewDay`, `groupedWithPrev` and `groupedWithNext`: same sender, same day, and less than 5 minutes apart. Grouped bubbles get tighter margins and squared corners, and the avatar shows only on the last bubble of a group.

Q: How do themes work? The `data-theme` attribute is set on `<html>`, and daisyUI themes and CSS variables restyle everything. The theme is saved in localStorage as `chat-theme`. `LEGACY_THEME_MAP` migrates old names (for example `dracula` → `vampire`). The favicon is regenerated as an SVG data URL in the theme's `--color-primary`.

Q: How is the layout responsive? The sidebar is full-width on mobile and hidden (`hidden lg:flex`) while a chat is open; back arrows clear `selectedUser`. On large screens the sidebar is a fixed `23rem`.

Q: How does auto-scroll work? A `messageEndRef` div sits at the bottom, and `scrollIntoView({behavior:'smooth'})` runs in an effect whenever `messages` changes.

Q: What are the skeletons for? `SideBarSkeleton` and `MessageSkeleton` show while loading, which improves perceived performance and avoids layout shift.

4. Technology fundamentals they may drill into

Interviewers often move from your project to the concepts underneath it. Each answer below ties back to Prism.

4.1 React 19

- **Virtual DOM / reconciliation?** React builds an element tree, diffs it against the previous one, and applies the smallest set of DOM changes. `key` props (for example `key={contact._id}` in the Sidebar) let it match list items across renders.
- **Why not use the array index as a key?** Reordering or inserting would attach state and DOM to the wrong item. Prism uses `_id` and `deviceId`.
- **`useEffect` dependency array?** The effect re-runs when a dependency changes. `[]` runs once after mount. Returning a function cleans up, as the Escape-key listener in `ChatContainer` does.
- **Why does StrictMode double-invoke effects in development?** To expose missing cleanups. `checkAuth` running twice is harmless; socket listeners are protected by `socket.off`.
- **`useMemo` vs `useCallback`?** `useMemo` caches a value (the grouped `rows`); `useCallback` caches a function. Use them only when a computation is expensive or a stable identity matters.

- **`useRef`?** A mutable box that doesn't trigger re-renders. Used for `messageEndRef` (scrolling) and `fileInputRef` (clicking the hidden file input, resetting its value).
- **Controlled vs uncontrolled inputs?** The text inputs are controlled (`value` + `onChange`). The file input is effectively uncontrolled and reached through a ref.
- **Lifting state vs a global store?** Chat and auth state are shared by Sidebar, ChatHeader, ChatContainer and the modals, so a store avoids prop drilling.
- **What's new in React 19?** Actions, `useActionState`, `useOptimistic`, the `use()` hook, ref as a prop, and the React Compiler. A good answer: "`useOptimistic` would fit my send-message flow."
- **What causes a re-render?** A state or prop change, a parent re-render, or a context change. With Zustand, a component re-renders when the selected slice changes. Prism destructures the whole store, so any change re-renders; selectors like `useChatStore(s => s.messages)` would cut that down.
- **Conditional rendering and routing guards?** `authUser ? <HomePage/> : <Navigate to='/login'/>`.
- **Code splitting?** `import('./useChatStore.js')` is a dynamic import; `React.lazy` could split the pages too.

4.2 Zustand

- **Why Zustand over Redux or Context?** No providers, no reducers or actions boilerplate, about 1 KB. Selectors avoid the problem where Context re-renders everyone. `getState()` works outside React, which the socket handler needs.
- **How does `create((set, get) => ...)` work?** `set` merges partial state (shallow); `get` reads the current state. Async actions are just async functions.
- **How does one store read another?** `useAuthStore.getState().authUser` inside the chat store.
- **Pitfall: stale state in closures.** The `newMessage` handler calls `get()` on every event to read the current `selectedUser`, rather than capturing it when the listener was registered.
- **Persistence?** It's done by hand with localStorage in `useThemeStore`; the `persist` middleware is the built-in option.

4.3 JavaScript / browser

- **Event loop?** The call stack, then the microtask queue (promises), then the macrotask queue (timers, I/O). Prism uses `setTimeout(..., 50)` to read CSS variables after the theme attribute has been applied.

- **localStorage vs sessionStorage vs cookies vs IndexedDB?** localStorage is about 5 MB, synchronous, per-origin and permanent. sessionStorage is per-tab. Cookies are sent automatically and can be httpOnly. IndexedDB is async, large, and can hold non-extractable CryptoKeys.
- **What's a data URL / base64?** The binary is encoded as text (about 33% larger); used for image previews and the dynamic favicon.
- **FileReader and canvas?** `readAsDataURL`, then load into an `Image`, `drawImage` onto a canvas, and `toDataURL('image/webp', 0.7)`.
- **`crypto.randomUUID()`?** A CSPRNG v4 UUID, used for the device id. The `Math.random` fallback isn't cryptographically secure.
- **Promises / async-await error handling?** Every store action uses `try/catch/finally`, setting the loading flag in `finally`.
- **CORS?** The browser enforces the same-origin policy. In development the frontend runs on :5173 and the API on :5001, so the server sends `Access-Control-Allow-Origin: http://localhost:5173` and `Allow-Credentials: true` (a wildcard isn't allowed when credentials are sent). In production both are on the same origin, so CORS isn't involved.

4.4 Node.js and Express 5

- **Why is Node good for chat?** Its non-blocking I/O and event loop handle many idle connections cheaply. CPU-heavy work (like bcrypt) runs in the libuv thread pool.
- **What's middleware?** A function `(req, res, next)` in a pipeline: `cors` → `express.json` → `cookieParser` → router → `ProtectedRoute` → controller.
- **What changed in Express 5?** Rejected promises from async handlers go to the error handler automatically, and path syntax changed (`*` must be named, hence the regex `app.get(/.*/)` for the SPA fallback). Some removed methods.
- **Why does Socket.IO share the HTTP server?** `http.createServer(app)` lets Express and Socket.IO listen on one port; the WebSocket upgrade happens on the same server.
- **ES modules?** `"type": "module"`, so `import` / `export`. `__dirname` doesn't exist in ESM, which is why the code uses `path.resolve()`.
- **dotenv?** It loads `.env` into `process.env`. Never commit it (`.gitignore`).
- **Why connect to the DB after `listen`?** The port opens quickly (good for Render health checks), but early requests could fail. Better: `await connectDB()` before listening, and exit if it fails.
- **Status codes used?** 200 OK, 201 Created (sign-up/send), 400 Bad Request, 401 Unauthorized, 404 Not Found, 500 Server Error. What's 403? Authenticated but not allowed.

- **REST principles?** Resources and HTTP verbs, stateless requests. Prism's `/auth/addContact` isn't very RESTful; better would be `POST /api/contacts` and `DELETE /api/contacts/:id`.
- **`nodemon`?** It restarts the server when files change in development (`npm run dev`).

4.5 MongoDB and Mongoose

- **Why MongoDB for chat?** A flexible schema, documents that map to JSON, horizontal scaling with sharding, and a managed free tier on Atlas.
- **SQL vs NoSQL here?** Chat has simple access patterns (one conversation, by time). Relational databases also work well (Discord started on MongoDB and moved to Cassandra, then ScyllaDB, for scale).
- **What's an index? What does `{senderId:1, receiverId:1}` speed up?** A B-tree for lookups. It helps the `$or` conversation query (each branch uses the index), but sorting by `createdAt` happens in memory. The best compound index is `{senderId:1, receiverId:1, createdAt:1}`. Bonus: a `conversationId` field turns this into a single-range query.
- **What's `.lean()`?** It returns plain JS objects instead of Mongoose documents: faster, no getters or `save`. Used in `ProtectedRoute`.
- **`.select('-password')`?** A projection that excludes the hash.
- **`$push`, `$pull`, `$set`, `$in`, `$ne`, `$or`?** All used: add a contact, remove a contact, mark read, sidebar users, exclude self, conversation query.
- **`findByIdAndUpdate(..., {new:true})`?** Returns the document after the update.
- **Embedding vs referencing?** Embed when the data is bounded and read together (devices). Reference when it's unbounded or shared (messages). The 16 MB document limit is the key fact.
- **`timestamps: true`?** Adds `createdAt` and `updatedAt` automatically.
- **`unique: true`?** It creates a unique index, not a validator. Duplicates also raise E11000, so the code checks `findOne` first.
- **Pagination?** Cursor-based: `find({..., createdAt: {$lt: cursor}}).sort({createdAt:-1}).limit(50)`, which is better than `skip` for large offsets.
- **Aggregation pipeline?** `$match` → `$group` → `$sort` → `$lookup`, used for unread counts or the last message per conversation.
- **Transactions?** Multi-document ACID on replica sets. Not needed here: every write touches a single document.

4.6 Authentication and JWT

- **JWT structure?** base64url(header).base64url(payload).signature (HMAC-SHA256 with `JWT_SECRET`). The payload is encoded, not encrypted, so never put secrets in it. Prism's payload is `{userId, iat, exp}`.
- **Sessions vs JWT?** Sessions are stored on the server and easy to revoke. JWTs are stateless and need no DB lookup, but revocation is hard. Prism looks up the user on every request anyway, so it pays for a DB read without getting revocation.
- **Access + refresh tokens?** A short access token (about 15 minutes) plus a long refresh token that's rotated and stored server-side.
- **bcrypt, salt, cost factor?** bcrypt is a slow, adaptive hash with a random 16-byte salt built in. Each +1 to the cost doubles the time; 10 is about 50–100 ms. Salt defeats rainbow tables; slowness defeats brute force. Argon2id is the modern recommendation.
- **Why not SHA-256 for passwords?** It's too fast: GPUs try billions per second.
- **Cookie flags?** `httpOnly` (no JS access), `secure` (HTTPS only), `sameSite=strict` (not sent cross-site), `maxAge`.
- **What attacks can still work?** XSS can still make requests as the user (it just can't read the cookie). There's no login rate limiting (brute force). The password policy is weak (6 characters).

4.7 Socket.IO

- **Handshake?** HTTP long-polling first, then an upgrade to WebSocket. A ping/pong heartbeat detects dead connections.
- **Rooms vs direct socket ids?** Rooms (`socket.join(userId)`) handle several sockets per user and scale with adapters. Prism emits to a single socket id.
- **Broadcast?** `io.emit` sends to all, `socket.broadcast.emit` to all except the sender, `io.to(room).emit` to a room.
- **Acknowledgements?** `socket.emit('evt', data, (ack) => ...)` confirms delivery.
- **Authenticating sockets?** In Prism, `userId` comes from the query string, which isn't verified (see section 6). The right way: `io.use((socket, next) => verify JWT from socket.handshake.headers.cookie)`.
- **Reconnection?** Automatic with backoff. After reconnecting you must re-fetch anything you missed, because events aren't buffered for disconnected clients (connection state recovery in v4.6+ helps for short gaps).
- **Scaling?** Use `@socket.io/redis-adapter` and sticky sessions (see section 5).

4.8 Tailwind v4, daisyUI and Vite

- **What does utility-first mean?** Compose small classes in markup. No naming problems and no dead CSS: v4 scans your sources and generates only the classes you use.
- **What's new in Tailwind v4?** CSS-first config (`@theme`), the `@tailwindcss/vite` plugin, the Oxide engine, native cascade layers.
- **daisyUI?** Component classes and themes driven by CSS variables. Prism defines 9 custom glass themes.
- **Glassmorphism?** `backdrop-filter: blur()`, semi-transparent backgrounds, subtle borders. Performance cost: blur is GPU-heavy, especially on mobile.
- **Vite vs CRA/Webpack?** Native ESM dev server (no bundling in development), so startup and HMR are instant. Production builds use Rollup (Rolldown in Vite 8). `import.meta.env.MODE` and `VITE_*` variables are exposed to the client, so never put secrets in `VITE_*`.

5. System design and scaling questions

The standard follow-up is "How would you scale this to 1M users?" Start from what Prism does today, name the bottleneck, then give the fix.

Bottlenecks in the current design, and their fixes

Today in Prism	Why it breaks at scale	Fix
`userSocketMap` is an in-memory object in one process	A second server instance can't see sockets on the first; a restart wipes presence	Redis adapter for Socket.IO plus presence in Redis (a set or a key with TTL per user); sticky sessions on the load balancer
One socket id per user	Multiple tabs or devices break delivery and presence	`socket.join(userId)` rooms; count connections per user before marking them offline
`getOnlineUsers` broadcasts the whole online list to everyone	O(N²) traffic: every connect sends N ids to N clients	Send presence only to a user's contacts, as deltas (`userOnline` / `userOffline`), with lazy fetches
The whole conversation loads at once (`getAllMessages`)	Megabytes of JSON and slow decryption for long chats	Cursor pagination (`before=<createdAt>`, limit 50) + infinite scroll + a virtualised list (react-window)

Today in Prism	Why it breaks at scale	Fix
The sort isn't covered by the index	In-memory sort, slow queries	Index `{senderId, receiverId, createdAt}` or a `conversationId + createdAt` index
The sidebar loads every received message to find senders and unread counts	Linear in history size	A `Conversations` collection (participants, lastMessage, unread per user) updated on send; or an aggregation
`/messages/users` returns every user	Data leak + huge payload	A server-side exact-match search endpoint, limit + rate limit
A single MongoDB cluster	Write throughput limits	Shard messages by `conversationId` (hashed); move old messages to cold storage
No queue between send and fan-out	Spikes overload the process	A message broker (Kafka/Redis Streams) between the API and the socket gateways
No offline delivery	Offline users miss notifications	Web Push / FCM / APNs workers; only the encrypted payload or a "new message" ping

Model answer: "Design WhatsApp-lite based on Prism"

1. **Clients** keep a persistent WebSocket to a **gateway** tier behind an L4/L7 load balancer with sticky sessions.
2. **Gateways** are stateless apart from their connections. They register `userId → gatewayId` in Redis on connect.
3. **Send path:** client → gateway (or REST) → **chat service** checks auth, assigns a message id (Snowflake/ULID for ordering), and writes to the **message store** (Cassandra/ScyllaDB partitioned by conversationId, clustered by time; or sharded MongoDB).
4. The chat service publishes to a **broker**. The receiver's gateway (found via Redis) pushes over its socket; if the receiver is offline, a **push notification** service sends a ping.
5. **Delivery states:** sent (server ack), delivered (receiving client ack), read (read event). Prism has sent + read today.
6. **E2EE stays client-side;** the server only sees ciphertext blobs. Add a prekey server for the Signal protocol.

7. **Media:** client encrypts → pre-signed URL → object storage (S3) + CDN; the key travels inside the message.
8. **Presence:** heartbeat to Redis with a TTL; fan out only to subscribed contacts.

Capacity estimate example (practise saying it)

Say 1M daily users, 40 messages a day each: 40M messages a day, about 460 per second on average and about 2,000 per second at peak. At around 500 bytes of ciphertext plus metadata per message, that's about 20 GB a day or 7 TB a year before replication. With 30% of users connected at once, that's 300K concurrent sockets; at about 50K per gateway node, 6 or more gateways.

Specific scaling questions

Q: How do you guarantee message order? Per conversation, order by a server-assigned monotonic id or timestamp. Today Prism sorts by `createdAt` from the Mongo server clock. Two messages in the same millisecond could tie, so an ObjectId `_id` sort is safer (it's roughly time-ordered).

Q: How do you avoid duplicate messages on retry? The client generates a `clientMessageId` (UUID); a unique index makes the send idempotent.

Q: Exactly-once delivery? In practice it's at-least-once delivery plus de-duplication on the client.

Q: How do you handle a reconnect after missing events? On reconnect, fetch messages since the last seen id. Prism re-fetches when you open a chat but doesn't sync on socket reconnect.

Q: Caching? Redis for user profiles and public keys, which are read on every chat open. A CDN for static assets and images. HTTP caching for the built JS bundle (Vite hashes filenames, so it can be cached for a long time).

Q: How would you add group chats? A `Conversation` model with `participants[]`, `type: 'dm' | 'group'`, and messages keyed by `conversationId`. Emit to room `conv:<id>`. For E2EE, sender keys (see 3.1).

Q: How would you add typing indicators? A `typing` socket event with throttling (once every 2–3 seconds); the client clears it after a timeout. It's ephemeral, so it's never stored.

Q: How would you add message edit/delete? Add `editedAt` and `deletedAt` fields and emit `messageUpdated`. With E2EE, an edit is a new encrypted payload. "Delete for everyone" is a tombstone.

Q: How would you add search? With E2EE the server can't index plaintext, so search has to run on the client over decrypted local history (IndexedDB plus a local index). That's a real trade-off to mention.

Q: Monolith or microservices? A monolith is the right choice at Prism's size. You'd split out gateways (connection-heavy) from the API (request-heavy) first, because they scale differently.

6. Security, testing, deployment and performance

6.1 Security audit of Prism (know these before they find them)

A strong candidate raises weaknesses before the interviewer does. Each row is a real gap in the current code, ordered by severity.

#	Issue	Where	Impact	Fix
1	Socket identity comes from an unverified `handshake.query.userId`	`Socket.js`	Anyone can connect as any userId: they receive that user's `newMessage` events (ciphertext + metadata), knock them offline, and fake presence	Verify the JWT cookie in `io.use()` middleware; take userId from the token
2	Public keys are distributed by the server with no verification	`publicKey` on User	The server or an attacker with DB access can swap keys and MITM conversations	Safety numbers / QR verification; warn on key change
3	Secret key is stored in plaintext localStorage	`useAuthStore.signup`, `setSecretKeyPage`	Any XSS steals the key, which decrypts everything (no forward secrecy)	Non-extractable CryptoKey in IndexedDB; encrypt the key at rest with a passphrase; strict CSP
4	Images are uploaded unencrypted; only the URL is encrypted	`useChatStore.sendMessage`	Anyone with the URL can view the image	Encrypt the bytes with `secretbox`; upload an opaque blob
5	Unsigned Cloudinary preset in the client	`VITE_CLOUDINARY_UPLOAD_PRESET`	Anyone can upload to the account (abuse, cost)	Signed uploads with a short-lived signature from the server
6	`GET /messages/users` returns every user with email + publicKey	`getAllUsers`	User enumeration, email harvesting	Exact-match search endpoint returning minimal fields + rate limit
7	No authorisation on `send/:id`	`sendMessage`	Anyone can message anyone (spam); `:id` isn't validated as an ObjectId (CastError → 500)	Validate ids; optional contact/block lists
8	No rate limiting	all routes	Brute-force login, spam	`express-rate-limit`, account lockout
9	JWT can't be revoked; device removal is enforced on the client	`logout`, `removeDevice`	A stolen or removed device's token lasts 7 days	`tokenVersion` or a session table checked in `ProtectedRoute`; per-device sessions

#	Issue	Where	Impact	Fix
10	`jwt.verify` throwing returns 500, not 401	`ProtectedRoute`	Wrong semantics; noisy logs	Catch `JsonWebTokenError` / `TokenExpiredError` → 401
11	50 MB JSON body limit	`index.js`	Memory DoS	1–10 MB; multipart for files
12	No input validation or sanitisation	controllers	NoSQL operator injection (for example `{"email": {"$ne": null}}` in login)	zod/joi validation, `express-mongo-sanitize`, enforce string types
13	Weak password policy (6 characters), email not normalised	`signup`	Weak accounts; duplicates by letter case	Longer minimum / zxcvbn check; lowercase + trim the email
14	No security headers or CSP	`index.js`	Raises the impact of XSS	`helmet()` with a strict CSP
15	Contact snapshots hold copies of public keys	`addContact`	Stale keys if a key ever changes	Store ids only; fetch public keys fresh

Q: Is React safe from XSS? Mostly. JSX escapes interpolated strings, and Prism doesn't use `dangerouslySetInnerHTML`. Remaining risks are `href` / `src` with `javascript:` URLs (the decrypted image URL is rendered as `src`, which should be validated to start with `https://res.cloudinary.com`) and third-party scripts.

Q: How do you prevent CSRF? `SameSite=strict` cookies are not sent on cross-site requests. The API takes JSON, which forces a CORS preflight for cross-origin calls. A CSRF token would be extra protection.

Q: What is NoSQL injection, and is Prism vulnerable? If `req.body.email` is an object like `{$gt: ""}`, `findOne({email})` matches the first user. bcrypt.compare would still fail without the password, so login is protected by accident, but other queries might not be. Validate types.

Q: How are secrets managed? `.env` in development and Render's environment variables in production. `VITE_*` variables end up in the client bundle, so only public values belong there.

Q: Bug to mention: environment variable name mismatch. `db.js` reads `MONGO_DB_URI`, but the README says `MONGODB_URI`. Someone who follows the README gets a failed connection. It's caught and only logged, so the server keeps running without a database.

6.2 Testing (Prism has no tests yet; say what you would add)

Q: How did you test it? Honest answer: manually, with two browsers or an incognito window logged in as different users, checking delivery, presence, unread counts and read receipts, plus inspecting MongoDB to confirm ciphertext. Then say what you would automate:

- **Unit (Vitest):** `crypto.js` round-trip encrypt/decrypt; a tampered ciphertext returns `[Encrypted Message]`; `verifySecretKey` with right and wrong keys; `formatDayLabel`, `isSameDay`; message grouping logic.
- **API integration (Jest/Vitest + Supertest + mongodb-memory-server):** sign-up validation, duplicate email, login with bad credentials, 401 without a cookie, send/fetch messages, mark-read.
- **Socket tests:** two `socket.io-client` instances; assert `newMessage` and `messagesRead` arrive.
- **Component (React Testing Library):** MessageInput disables send when empty; Sidebar online filter.
- **E2E (Playwright):** two browser contexts chatting; device unlock flow.
- **CI:** GitHub Actions running lint + test + build on every PR.

6.3 Deployment

Q: How is it deployed? On Render as one web service. Build: `npm run build` at the root installs backend and frontend dependencies and runs `vite build`. Start: `npm start` runs `node backend/src/index.js`. With `NODE_ENV=production`, Express serves `frontend/dist` and sends `index.html` for every other route so client-side routing works. MongoDB Atlas hosts the DB; Cloudinary hosts images.

Q: Why serve the frontend from Express rather than Vercel/Netlify? One origin means no CORS in production, the cookies just work (no third-party cookie problems), and there's one thing to deploy. Trade-off: no edge CDN for static files unless you put one in front.

Q: How does the client know the API URL? `import.meta.env.MODE === 'development' ? 'http://localhost:5001/api' : '/api'`, a relative URL in production. Socket.IO works the same way.

Q: What about Render free tier cold starts? The service sleeps after inactivity, so the first request takes about 30–50 seconds. Keep it awake with a paid instance or a health-check ping.

Q: Why does `secure` depend on `NODE_ENV`? In development, localhost runs over HTTP, so a `secure` cookie wouldn't be set. Render terminates HTTPS at its proxy.

Q: How would you containerise it? A multi-stage Dockerfile: build the frontend in a node image, copy `dist` + backend into a slim runtime, run `node src/index.js`. Docker Compose for local Mongo.

Q: Logging and monitoring? Today it's `console.log`. Better: structured logs (pino), error tracking (Sentry), uptime checks, metrics (socket count, message rate).

6.4 Performance

- **Done already:** client-side image compression (WebP, at most 1000 px); `.lean()` in auth middleware; the `{senderId, receiverId}` index; `useMemo` for grouping; skeletons; Vite production build with hashed, minified chunks; images on Cloudinary's CDN.
- **Next:** message pagination + list virtualisation; Zustand selectors to reduce re-renders; decrypt lazily or in a Web Worker for long histories; cache public keys; the aggregation-based sidebar; HTTP compression (`compression` middleware); lazy-load routes with `React.lazy`; Cloudinary transformations (`w_400,f_auto,q_auto`) for thumbnails.
- **Q: Why might the chat feel slow with 10,000 messages?** All of them are fetched, decrypted synchronously on the main thread, and rendered as DOM nodes. Fix it with pagination + virtualisation + a worker.
- **Q: Why isn't the send optimistic?** The message appears only after the POST resolves. With `useOptimistic` or a temporary id, it would appear immediately with a "sending" state, then reconcile or show an error.
- **Q: Memory or connection leaks?** `useChatStore.js` creates a second, unused `io(BASE_URL)` socket at module load, an extra anonymous connection per client. It should be removed.

7. Behavioural, project-story and "what would you improve" questions

Use STAR (Situation, Task, Action, Result) and keep each story under 2 minutes. The stories below come from this repo's real commit history ("E2EE", "key sharing", "real time contacts update", "Read unread messages", "UI changes suggested by ashmit", "mobile UI", "deployment changes", "finally working").

Project-story questions

Q: Why did you build this? I wanted to learn real-time systems and applied cryptography together. Most tutorial chat apps store plaintext; I wanted the server to see nothing, which forced me to think about key management, multi-device, and what E2EE can and can't hide.

Q: What was the hardest part? Multi-device with E2EE. The secret key lives only in the browser, so a second device can't decrypt anything. I solved it by letting users paste their secret key, which I verify locally by re-deriving the public key, then registering the device. I also added a device check on every load, so removing a device elsewhere wipes the key there.

Q: Tell me about a bug you fought. Deployment (the commits "deployment changes" → "finally working"). In production the SPA fallback and Socket.IO URL broke. Express 5 changed wildcard path syntax, so I used a regex route `/.*/`, and I switched the socket and API base URLs to relative paths when not in development, so one Render service serves both.

Q: Another bug story: duplicate socket listeners. Messages appeared twice and sounds played twice after reconnecting or re-rendering. The fix was `socket.off('newMessage')` before `socket.on`, plus guarding `connectSocket` when already connected.

Q: Tell me about feedback you received. A reviewer (Ashmit) suggested UI changes across three iterations. I took the feedback, prioritised readability and mobile layout, and shipped them in separate commits.

Q: What would you do differently? Authenticate the socket from day one, design a `Conversation` collection instead of computing sidebars from messages, encrypt image bytes rather than URLs, store keys in IndexedDB, and write tests for the crypto module first.

Q: What are you most proud of? The server genuinely stores only ciphertext. I can show the DB next to the UI, and tampered ciphertext is detected by Poly1305.

Q: How did you decide on the tech stack? MERN for one language across the stack and quick iteration. Socket.IO for reliability features. Zustand because Redux was overkill for 3 stores. TweetNaCl because it's audited and hard to misuse.

Q: How long did it take, and how did you work? Iterative: core chat → deployment → E2EE → contacts and key sharing → real-time contacts → read receipts → UI polish and mobile. Commit messages map to these phases. (Have your own honest dates ready.)

Q: Did you use AI tools? Answer honestly. A good framing: "I used them for UI iteration and boilerplate. I designed the crypto flow and verified every part myself, and I can explain each line."

Known bugs to raise yourself (shows ownership)

1. **Contact delete never works on the server:** the route is `/delete/:id`, but the controller reads `req.params.contactId`, which is undefined. The client removes the contact from local state anyway, so it reappears on refresh. Contacts are also stored as `_id` ObjectIds inside an untyped array while the param is a string, so `$pull` may not match without a cast.
2. **Last tab wins:** multiple tabs or devices break real-time delivery and presence (one socket id per user).
3. **Stray socket** created at module load in `useChatStore.js`.

4. **Environment variable mismatch** `MONGO_DB_URI` vs README `MONGODB_URI`; the DB connection error is swallowed.
5. **Sign-up stores the secret key before the server confirms.** A failed sign-up leaves an orphan key; a retry generates a new key and overwrites it (harmless, but untidy).
6. **A new sender doesn't appear in the sidebar live.** A message from someone not yet in the sidebar bumps the unread count, but the sidebar list isn't refreshed until reload.
7. **Login returns 201** instead of 200; `ProtectedRoute` returns 500 for invalid or expired tokens.
8. **Logout doesn't unregister the device** or clear the local secret key. That's arguably intended (fast re-login), but on a shared computer it's a risk.
9. **The image fallback path:** if the Cloudinary upload fails, the message still sends without the image after showing a toast.
10. **`isRead` updates** on the sender's side compare `msg.receiverId === receiverId`; this works because both are strings in JSON, but there's no per-message delivered state.

Trade-offs you made (and why)

Decision	Benefit	Cost
REST for send, socket for push	Status codes, auth middleware reuse, simple errors	An extra hop; two channels to reason about
Static key pair per user (no ratchet)	Simple, works across devices with one key	No forward secrecy; one key leak exposes all history
User-managed secret key (copy/paste)	The server never holds keys	Bad UX; losing the key loses history
localStorage for keys	Simple, synchronous	Readable by XSS
Embedded contacts/devices	No joins, one read	Stale snapshots, untyped array
Single Render service	No CORS, simple cookies, one deploy	No independent scaling, no edge CDN
Client-side Cloudinary upload	Large files bypass the server	Unsigned preset abuse; image not E2EE
Zustand without selectors	Less code	Extra re-renders

Roadmap answer ("If you had 2 more weeks...")

1. Socket auth middleware + rooms per user (fixes security issue #1 and the multi-tab bug).
2. Fix contact delete; move contacts to ObjectId refs.
3. Message pagination + `conversationId` + compound index.
4. Encrypted attachments with `secretbox`; signed uploads.
5. Tests (crypto unit tests, Supertest API tests, Playwright two-user E2E) + GitHub Actions CI.
6. helmet, rate limiting, zod validation, 401 handling.
7. Typing indicators, delivered ticks, optimistic sends, push notifications.
8. Key fingerprint verification; later the Signal protocol (X3DH + Double Ratchet) for forward secrecy.

General behavioural questions to prepare

- Tell me about yourself (end with this project as the hook).
- A time you learned something difficult quickly (cryptography for E2EE).
- A time you disagreed with feedback, or a trade-off you had to defend.
- A time something failed in production (deployment).
- How you prioritise features against polish.
- How you would onboard a teammate to this codebase (folder map, flows, env setup in the README).
- Questions to ask them: how their team handles real-time infrastructure, testing culture, what a first project would be.

8. Rapid-fire cheat sheet (read 10 minutes before the interview)

Question	One-line answer
Crypto primitive?	`nacl.box` = X25519 key exchange + XSalsa20 cipher + Poly1305 MAC
Key size / nonce size?	32-byte keys, 24-byte random nonce per message
Where is the secret key?	Browser localStorage, `<email>_secret_key`; never sent to the server
Where is the public key?	`User.publicKey` (base64) in MongoDB

Question	One-line answer
How does the sender read their own message?	The DH shared secret is symmetric: my secret + their public key
Tampered ciphertext?	`box.open` returns null → shows "[Encrypted Message]"
Verify a pasted key?	Re-derive the public key with `keyPair.fromSecretKey`, compare
Forward secrecy?	No; next step is the Double Ratchet
MITM risk?	The server distributes keys; fix with safety numbers
Image E2EE?	Only the URL is encrypted; the file on Cloudinary is plaintext
Image optimisation?	Canvas resize to at most 1000 px, WebP q 0.7
Real-time transport?	Socket.IO on the same HTTP server as Express
Socket events?	`getOnlineUsers`, `newMessage`, `messagesRead`
Online map?	In-memory `userSocketMap` {userId: socketId}
Why REST to send?	Status codes, auth middleware, simple errors; socket only pushes
Multi-tab bug?	One socket id per user; fix with rooms
Socket auth?	None (userId from query); fix with JWT in `io.use`
Auth?	JWT `{userId}` 7 days in an httpOnly, SameSite=strict, secure cookie
Password hashing?	bcrypt, 10 salt rounds
Protected route?	`ProtectedRoute` middleware → `req.user` (no password, `.lean()`)
Logout?	Cookie `maxAge: 0`; the JWT isn't revoked
State management?	Zustand: auth, chat, theme stores
Unread counts?	Server: count `isRead:false` per sender; client: increment on `newMessage`
Read receipts?	PUT `mark-read/:id` → `updateMany` → emit `messagesRead`

Question	One-line answer
Sidebar?	Contacts ∪ people who messaged me (Set de-duplication)
Index?	`{senderId:1, receiverId:1}`; add `createdAt`
Why messages in their own collection?	Unbounded; 16 MB document limit
Devices?	UUID in localStorage + UA parsing; embedded in User
Device revocation?	Enforced on the client; JWT still valid; key not rotated
Deployment?	Render, one service; Express serves `frontend/dist` + SPA fallback `/.*/`
CORS?	Development only (5173 → 5001), credentials true
Body limit?	50 MB (too high)
Themes?	9 daisyUI themes via `data-theme`; dynamic SVG favicon
Message grouping?	Same sender, same day, under 5 minutes apart
Biggest known bug?	Contact delete reads the wrong param (`contactId` vs `:id`)
Top 3 improvements?	Socket auth + rooms; pagination + Conversation model; encrypted attachments + tests
Scale to 1M?	Redis adapter + sticky sessions, sharded messages, broker, push service, presence deltas

Words to use (they signal depth)

Authenticated encryption, nonce reuse, Diffie-Hellman, forward secrecy, key distribution / MITM, trust on first use, metadata leakage, httpOnly / SameSite, stateless vs stateful auth, token revocation, idempotency, at-least-once delivery, cursor pagination, compound index, denormalisation, sticky sessions, pub/sub adapter, fan-out, presence deltas, optimistic UI, list virtualisation.

Live-demo checklist

- [ ] Two browsers (one incognito) logged in as different users
- [ ] Send text and an image; show instant delivery, sound, and the unread badge
- [ ] Open the chat; show the read receipt turning on for the sender
- [ ] Open MongoDB Atlas; show `text` as ciphertext with `textNonce`
- [ ] Log in on a third browser; show the "Unlock this device" flow and the Devices list
- [ ] Switch themes; show the mobile layout in responsive mode
- [ ] Have the Render cold start warmed up beforehand

Files to have open during a code walkthrough

`frontend/src/lib/crypto.js` · `frontend/src/stores/useChatStore.js` · `frontend/src/stores/useAuthStore.js` · `backend/src/lib/Socket.js` · `backend/src/controllers/message.controllers.js` · `backend/src/middleware/auth.middleware.js` · `backend/src/models/*.js` · `frontend/src/pages/HomePage.jsx` (device check) · `frontend/src/pages/setSecretKeyPage.jsx`